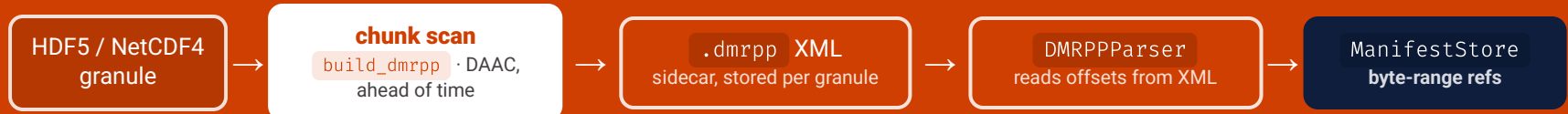
developmentSEED

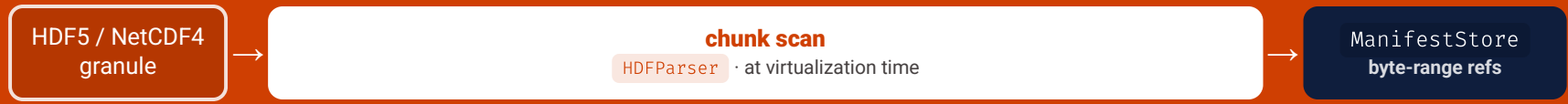
TWO PATHS, ONE RESULT

Both build the same ManifestStore

DMR++ PATH



DIRECT PATH



The chunk scan is in **both** paths. DMR++ doesn't skip it, it runs it **earlier as part of the DAAC ingestion pipeline**

developmentSEED

SAME SCAN, DIFFERENT TIME

DMR++ generation and direct parsing call the same HDF5 routine

BES · C++ dmrpp_module/build_dmrpp_util.cc

```cpp
if (H5Dchunk_iter(dataset, H5P_DEFAULT,
                  &chunk_cb, &chunk_udata) < 0) {
    throw BESInternalError("H5Dchunk_iter fails.", ...);
}
for (unsigned int i = 0; i < num_chunks; ++i) {
    dc->add_chunk(byte_order,
                  chunk_udata.chunk_sizes[i],
                  chunk_udata.chunk_addrs[i], ...);
}
```

VIRTUALIZARR · PYTHON parsers/hdf/hdf.py

```python
chunk_infos: list = []
dsid.chunk_iter(chunk_infos.append)  # H5Dchunk_iter

for chunk_info in chunk_infos:
    key = get_key(chunk_info)
    paths[key] = filepath
    offsets[key] = chunk_info.byte_offset
    lengths[key] = chunk_info.size
```

The quoted **4–20 ms per file** parsing time for DMR++ (VirtualiZarr PR #113) is the time to read XML *after* this loop already ran on the DAAC's ingestion. Direct parsing runs it at virtualization time: **1–5 s** median per file (and there is a list of upcoming improvements).

developmentSEED

THE WHOLE BILL

The scan is a small line item

DMR++ path

Chunk scan, every granule	**paid once, by the DAAC**
Store and serve a sidecar per granule	ongoing
Rebuild sidecars when BES adds fields (e.g. deflate level)	per collection
Find and rebuild stale or wrong sidecars	per collection
HDF fallback for collections without DMR++	~85% of collections
Keep the parser in step with format drift	VirtualiZarr + pydap
Maintain the generator in BES (C++)	OPeNDAP

Direct path

Chunk scan, every granule	**paid once, at virtualization**
Store the result in Icechunk	never re-scanned
Maintain the parser	VirtualiZarr (Python)

Once the store is written, nobody re-parses. The scan is a **one-time cost either way**

developmentSEED

DUPLICATION

Every sidecar dmr++ repeats the same metadata

MUR SST L4 · PO.DAAC one daily sidecar, 2025-06-01 vs 06-02

87% repeated verbatim from the day before | 13%

dtype, dims, 128 attributes, codec chains, chunk grid & positions

new: offsets, lengths, dates

Across all 8,914 granules	Copies
Attributes	> 1 million
Codec chains (shuffle + deflate)	~60,000
Sidecar XML, 115–213 KB each	~1–2 GB

And the copies **don't agree**: 2010 and 2018 sidecars (BES 3.20.13) have no deflate level; 2025 sidecars (BES 3.21.0) say **level 7**.

WHERE IT LIVES

DMR++ · ONE COPY PER GRANULE



× every granule

ICECHUNK · ONE COPY PER ARRAY

`zarr.json`
dtype, shape, codecs, attrs: stored once



chunk manifest · offset, length per chunk

Codec chain, dtype and attributes become **one array definition**. Only chunk refs grow with granule count.

development SEED

A SECOND-HAND COPY OF THE FILE

The sidecar can be wrong; the file can't

Problem	Where	Fixable in VirtualiZarr?
Missing deflate level. Added to `build_dmrpp` Sep 2023; older MUR SST, SWOT, ICESat sidecars lack it	OPENDAP/bes#954	No: DAAC rebuild
Wrong dtype. 64-bit time described as `Float32` ; decodes to negative timedeltas	VirtualiZarr #904	No: DAAC rebuild
Out of date. "In some cases they are not up to date"	earthaccess #908	No: DAAC rebuild
Format drift. New `<block/>` elements for HDF4 as an example	#85, #216	Every BES release

~15%

of NASA Earthdata collections have DMR++ that is **both downloadable and parseable** by VirtualiZarr (Openscapes, Aug 2025).

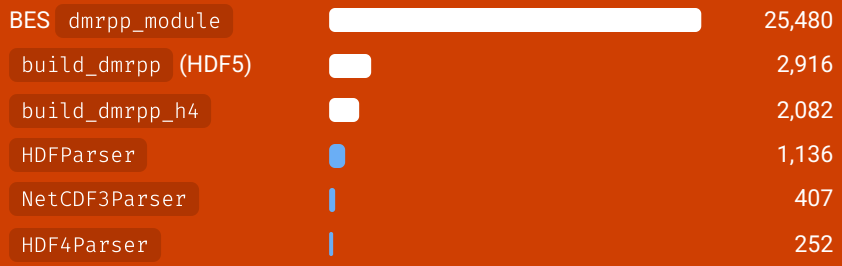
So any pipeline needs the VirtualiZarr HDF parser anyway. You maintain **both**.

development SEED

MAINTAINABILITY

Modular parsers vs. a C++ server module

LINES OF CODE (non-test, Oct 2026)



`build_dmrpp` links BES dispatch, libdap, the NGAP container, the HDF5 handler, OpenSSL and libxml2, and ships inside a **460k-line** C/C++ server. Format changes land on BES's release cycle.

HDF4 is where we need OPeNDAP. A more robust `HDF4Parser` needs their support: they hold the deep HDF4 knowledge (`hdf4_handler`, `build_dmrpp_h4`).

EVERY VIRTUALIZARR PARSER virtualizarr/parsers/typing.py

```python
class Parser(Protocol):
    def __call__(
        self,
        url: str,
        registry: ObjectStoreRegistry,
    ) → ManifestStore: ...
```

- **One small interface.** HDF5, HDF4, NetCDF3, FITS, ENVI, Zarr and Kerchunk all plug in the same way
- **Fixes land in one repo.** Fill values, var-length strings, links, chunk-grid shape: all fixed in 2026
- **Easy to contribute.** Python, `pip install`, tested on real files; Rust rewrite proposed (#1112)

developmentSEED

PERFORMANCE

Direct parsing is closing the gap from its own side

Change	Before	After	Status
Single B-tree walk instead of three (PR #1130), 1M-chunk file	56,195 requests · 140 MiB	**18,736 requests · 47 MiB**	Merged
`_dataset_chunk_manifest` rewrite (PR #1137)	2.34 s CPU · 728 MiB RSS	**0.79 s CPU · 244 MiB RSS**	Open
Read-ahead of chunk index over object storage (PR #1141), 8–12k chunks/dataset	15–36 s	**1.9–3.1 s**	Open
Same, 137 GB file with 196,608 chunks	> 15 min	**28 s**	Open
Rust rewrite of the HDF5 parser (#1112)		Proposed	Design

A few seconds per granule, once, then stored in Icechunk. That's the whole "cost" DMR++ saves.

developmentSEED

RECOMMENDATION

Parse the file; treat DMR++ as an optional cache

1 Default to direct parsing

Use `HDFParser` for HDF5/NetCDF4. It covers every granule, reads codecs and dtypes from the file, and its bugs get fixed in one repo.

2 Virtualize once, store in Icechunk

Pay the chunk scan one time per granule and never repeat it. That removes the only advantage a pre-built sidecar has.

3 DMR++ only as an opt-in cache

Only when the sidecar is newer than the granule and from a recent `build_dmrpp`. Validate a sample against `HDFParser` first.

4 Invest in parsers, not sidecars

Put effort into VirtualiZarr's modular parsers (incl. `HDF4Parser`), where every fix helps every collection.

development SEED

THANK YOU

Scan once. Store it. Own the code that reads it.

Sean Harkins · Development Seed

github.com/zarr-developers/VirtualiZarr · github.com/OPENDAP/bes